

Music Scheduler - Complete Installation Guide

This guide covers the complete installation and setup of the Music Scheduler application including the database backup/restore system.

Prerequisites

System Requirements

- **Operating System:** Ubuntu 20.04+ (or compatible Linux distribution)
- **Python:** 3.8 or higher
- **Database:** MySQL 8.0 or MariaDB 10.5+
- **Web Server:** Nginx (recommended)
- **Memory:** Minimum 2GB RAM
- **Storage:** Minimum 10GB disk space
- **Git:** For version control and deployments

Required Software

```
# Update system packages
sudo apt update && sudo apt upgrade -y

# Install required packages
sudo apt install -y python3 python3-pip python3-venv mysql-server nginx git curl

# Install Node.js for frontend (if needed)
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Installation Steps

1. Clone Repository

```
# Clone the repository
git clone https://github.com/dfultonthebar/music-scheduler.git
cd music-scheduler

# Make scripts executable
chmod +x scripts/*.sh
```

2. Database Setup

Install MySQL (if not already installed)

```
sudo apt install mysql-server
sudo mysql_secure_installation
```

Create Database and User

```
# Connect to MySQL as root
sudo mysql -u root -p

# Create database and user (run these SQL commands)
CREATE DATABASE music_scheduler CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'music_user'@'localhost' IDENTIFIED BY 'music_pass';
GRANT ALL PRIVILEGES ON music_scheduler.* TO 'music_user'@'localhost';
GRANT SELECT, LOCK TABLES ON music_scheduler.* TO 'music_user'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

3. Python Environment Setup

```
# Create virtual environment
python3 -m venv venv
source venv/bin/activate

# Upgrade pip
pip install --upgrade pip

# Install Python dependencies
pip install -r requirements.txt

# Install additional packages if needed
pip install flask flask-session flask-bcrypt flask-cors mysql-connector-python python-dotenv
```

4. Environment Configuration

```
# Copy environment template
cp .env.example .env # If example exists, or create new .env file

# Edit environment variables
nano .env
```

.env file contents:

```
# Flask Configuration
FLASK_APP=app.py
FLASK_ENV=production # Change to 'development' for dev mode
FLASK_DEBUG=False    # Change to True for dev mode
SECRET_KEY=your-secret-key-here-change-this

# Database Configuration
DB_HOST=localhost
DB_USER=music_user
DB_PASSWORD=music_pass
DB_NAME=music_scheduler

# Session Configuration
SESSION_TYPE=filesystem
SESSION_FILE_DIR=./sessions
```

5. Database Schema Setup

```
# Run database migrations/setup
python3 -c "
import mysql.connector
from mysql.connector import Error
import os
from dotenv import load_dotenv

load_dotenv()

try:
    connection = mysql.connector.connect(
        host=os.getenv('DB_HOST'),
        user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'),
        database=os.getenv('DB_NAME')
    )
    cursor = connection.cursor()

    # Add your table creation SQL here
    # This is just an example - adjust based on your schema
    cursor.execute('SHOW TABLES;')
    tables = cursor.fetchall()
    print('Connected to database successfully!')
    print('Tables:', tables)

except Error as e:
    print(f'Database connection error: {e}')
finally:
    if connection.is_connected():
        cursor.close()
        connection.close()
"
```

6. Frontend Setup (if applicable)

```
# Install Node.js dependencies
npm install

# Build frontend assets
npm run build
```

7. Create Required Directories

```
# Create necessary directories
mkdir -p database_backups uploaded_backups sessions logs static/uploads

# Set proper permissions
chmod 755 database_backups uploaded_backups sessions logs
chmod 644 .env
```

8. Test Installation

```
# Test database connection
python3 -c "
import sys
sys.path.append('.')
from app import get_db_connection
conn = get_db_connection()
if conn:
    print('✅ Database connection successful')
    conn.close()
else:
    print('❌ Database connection failed')
"

# Test Flask application
python3 app.py &
APP_PID=$!
sleep 3

# Test health endpoint
curl -f http://localhost:5000/api/health
if [ $? -eq 0 ]; then
    echo "✅ Flask application is running"
else
    echo "❌ Flask application failed to start"
fi

# Stop test application
kill $APP_PID
```



Backup System Setup

1. Initialize Backup System

```
# Create initial backup to test system
./scripts/backup_db.sh

# Verify backup was created
ls -la database_backups/
```

2. Setup GitHub Integration

```
# Configure git user (replace with your details)
git config user.name "Your Name"
git config user.email "your-email@example.com"

# Setup GitHub authentication
# Option 1: Personal Access Token
git config credential.helper store
# You'll be prompted for credentials on first push

# Option 2: SSH Key (recommended for servers)
ssh-keygen -t ed25519 -C "your-email@example.com"
# Add ~/.ssh/id_ed25519.pub to your GitHub account
git remote set-url origin git@github.com:dfultonthebar/music-scheduler.git
```

3. Setup Automatic Backups

```
# Add cron job for daily backups at 2:00 AM
(crontab -l 2>/dev/null; echo "0 2 * * * $(pwd)/scripts/backup_db.sh >> /tmp/
music_scheduler_backup_cron.log 2>&1") | crontab -

# Verify cron job was added
crontab -l
```

Web Server Configuration

Nginx Configuration

```
# Create Nginx configuration
sudo nano /etc/nginx/sites-available/music-scheduler
```

Nginx configuration file:

```
server {
    listen 80;
    server_name your-domain.com; # Replace with your domain

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static {
        alias /path/to/music-scheduler/static;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

Enable the site:

```
# Enable site
sudo ln -s /etc/nginx/sites-available/music-scheduler /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Systemd Service (Recommended for Production)

```
# Create systemd service file
sudo nano /etc/systemd/system/music-scheduler.service
```

Service file contents:

```
[Unit]
Description=Music Scheduler Flask Application
After=network.target mysql.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/path/to/music-scheduler
Environment=PATH=/path/to/music-scheduler/venv/bin
ExecStart=/path/to/music-scheduler/venv/bin/python app.py
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

Enable and start service:

```
sudo systemctl daemon-reload
sudo systemctl enable music-scheduler
sudo systemctl start music-scheduler
sudo systemctl status music-scheduler
```



Security Setup

1. Firewall Configuration

```
# Enable UFW firewall
sudo ufw enable
sudo ufw allow 22      # SSH
sudo ufw allow 80      # HTTP
sudo ufw allow 443     # HTTPS (if using SSL)
```

2. SSL Certificate (Recommended)

```
# Install Certbot
sudo apt install certbot python3-certbot-nginx

# Get SSL certificate
sudo certbot --nginx -d your-domain.com
```

3. Database Security

```
# Secure MySQL installation
sudo mysql_secure_installation

# Backup MySQL configuration
sudo cp /etc/mysql/mysql.conf.d/mysqld.cnf /etc/mysql/mysql.conf.d/mysqld.cnf.backup
```



Monitoring Setup

1. Log Rotation

```
# Create logrotate configuration
sudo nano /etc/logrotate.d/music-scheduler
```

Logrotate configuration:

```
/path/to/music-scheduler/logs/*.log {
    daily
    missingok
    rotate 30
    compress
    delaycompress
    notifempty
    copytruncate
}
```

2. Health Monitoring

```
# Create health check script
cat > /path/to/music-scheduler/check_health.sh << 'EOF'
#!/bin/bash
response=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:5000/api/health)
if [ "$response" != "200" ]; then
    echo "$(date): Health check failed - HTTP $response" >> /path/to/music-scheduler/
logs/health.log
    # Add notification logic here (email, Slack, etc.)
fi
EOF

chmod +x /path/to/music-scheduler/check_health.sh

# Add health check cron job (every 5 minutes)
(crontab -l 2>/dev/null; echo "*/5 * * * * /path/to/music-scheduler/check_health.sh")
| crontab -
```



Post-Installation Checklist

- [] Database connection working
- [] Flask application starts without errors
- [] Web interface accessible
- [] Admin login working
- [] Backup system functional
- [] Automatic backups scheduled
- [] GitHub integration configured
- [] SSL certificate installed (if applicable)
- [] Firewall configured
- [] Monitoring in place
- [] Log rotation configured

Troubleshooting

Common Issues

1. Database Connection Issues

```
# Check MySQL service
sudo systemctl status mysql

# Test database connection
mysql -u music_user -p -h localhost music_scheduler
```

2. Flask Application Won't Start

```
# Check Python dependencies
pip list

# Check for syntax errors
python3 -m py_compile app.py

# Check logs
tail -f logs/flask.log
```

3. Backup Script Fails

```
# Check mysqldump availability
which mysqldump

# Test manual backup
mysqldump -u music_user -p music_scheduler > test_backup.sql
```

4. Nginx Configuration Issues

```
# Test Nginx configuration
sudo nginx -t

# Check Nginx logs
sudo tail -f /var/log/nginx/error.log
```

Maintenance

Regular Maintenance Tasks

1. **Update System Packages:** `sudo apt update && sudo apt upgrade`
2. **Monitor Disk Space:** `df -h`
3. **Check Application Logs:** `tail -f logs/*.log`
4. **Verify Backups:** Test restore functionality monthly
5. **Update Dependencies:** `pip list --outdated`
6. **Security Updates:** Keep all software updated

Backup Maintenance

1. **Test Restore Process:** Monthly restoration tests
2. **Clean Old Backups:** Automatic cleanup after 30 days

3. **Monitor Backup Size:** Ensure adequate disk space
 4. **Verify GitHub Sync:** Check that backups are pushed to GitHub
-

Installation Date: __

Installed By: __

Version: 1.0

Last Updated: August 13, 2025